Lab 5: Static Code Analysis

Name: Manasvi GV
SRN: PES1UG24AM156
Team: SE_Team_05
Program: event_manager.py

Objective

The aim of this lab was to run Pylint, Bandit and Flake8 on the event_manager.py program, find the issues they report, fix them, and run the tools again to make sure the issues are gone.

About the program

event_manager.py is a small booking system for events. It creates events with a venue and capacity, registers guests until the event is full, shows the seats left, picks a lucky draw winner, marks events as upcoming or completed, and saves the events to a file.

Results

Before fixing, Pylint gave a score of 4.64/10 with 30 messages. Bandit found 4 issues (1 high, 1 medium and 2 low severity) and Flake8 found 4 style issues.

After fixing, Pylint gave 10.00/10, Bandit found no issues and the Flake8 report was empty.

Bug found by running the program

Before using the tools I ran the program. The output said "Alumni Dinner is full, cannot add Meena" and showed 0 seats left for Alumni Dinner, even though no one had registered for it. The reason was guests=[] in the createEvent function. A default list in Python is created only once, so both events were sharing the same guest list. Pylint only reported this as W0102 (dangerous default value). I changed the default to None and created a new list inside the function. After this Meena was registered and Alumni Dinner showed 1 seat left.

Bandit issues and fixes

1. B605, high severity, line 56: os.system was used to run the lp command, with the event name added into the command string. Someone could pass an event name with extra shell commands in it. I replaced it with subprocess.run and passed the arguments as a list so no shell is used. I also check the event name with a regex before using it.

2. B301, medium severity, line 51: pickle.load was used to load the saved events. Loading a pickle file that has been changed by someone else can run code. I changed the program to save and load the events using json instead.

3. B403, low severity, line 2: this was the import of pickle. It was removed when I switched to json.

4. B311, low severity, line 72: random.choice was used for the lucky draw. I changed it to secrets.choice.

After switching to subprocess, Bandit showed two low warnings for importing and calling subprocess. Since no shell is used and the input is checked, I marked these two lines with a nosec comment and wrote the reason above them.

Pylint issues and fixes

1. W0102, line 9: mutable default argument guests=[]. This is the bug explained above. Fixed by using None as the default.

2. W0613, line 39: the argument sortOrder in guestReport was never used. I made it work, so "name" sorts the guests from A to Z and "reverse" sorts them from Z to A.

3. R1705, lines 33 to 36: there was an unnecessary else after return in eventStatus. I removed the else.

4. C0103, lines 9 to 59: function and argument names were in camelCase. I renamed them to snake_case, for example createEvent to create_event and eventName to event_name, and changed all the calls in main.

5. C0114 and C0116: the module and functions had no docstrings. I added a docstring for the module and for every function.

I also put the call to main() inside if __name__ == "__main__" so the demo does not run when the file is imported.

Flake8 issues and fixes

1. E231, line 61: missing space after the comma in "Rooftop",2. Added the space.

2. E225, line 66: missing spaces around = in left=seatsLeft(...). Added the spaces.

3. E261, line 75: only one space before the inline comment. Changed it to two spaces.

4. E305, line 79: only one blank line after the last function. Changed it to two blank lines.

Commits

I made a separate commit for each tool. The first commit fixed the Bandit issues, the second fixed the Pylint issues and the third fixed the Flake8 issues. After that I added the before and after reports, issues.md and reflection.md.

Reflection

1. Which issues were the easiest and hardest to fix?

The Flake8 issues and the naming and docstring warnings from Pylint were the easiest, since the tools give the exact line and the fix is simple. The hardest one was the mutable

default argument. The Pylint message was short and I understood the problem only after running the program and seeing the wrong seat count. Replacing os.system also took some time because the function still had to work.

2. Did the tools report any false positives?

Yes. After my fix, Bandit still warned about subprocess even though no shell is used and the event name is checked, so I marked those lines with nosec. Also, many Pylint messages like missing docstrings are about style and are not actual bugs.

3. How would you use these tools in your development workflow?

I would run Flake8 and Pylint in the editor while writing code and add all three tools as pre-commit hooks. In CI, the build could fail if the Pylint score is below 9.5 or if Bandit finds any high or medium issue.

4. What improvements did you see after the fixes?

The seat counts and full messages are now correct, the shell injection and pickle problems are removed, and the code is easier to read because of snake_case names and docstrings. The guest report now uses its sort order argument and the file can be imported without running the demo.